

HRMS

Human Resource Management System

Technology Design Overview

For Client Review | Non-Technical Summary

Document Version	v1.0
Date	July 2026
Status	For Review & Approval
Classification	Confidential

Catalog

1. Purpose of This Document	3
2. Technology Approach	3
3. Technology Choices & Rationale	3
4. System Architecture Overview	5
5. Metadata-Driven Configuration	5
6. Security Design	7
7. Scalability & Performance	8
8. Hosting & Infrastructure	8
9. Development & Delivery Approach	8
10. Document Control	9

1. Purpose of This Document

This document provides a plain-language overview of the technology choices made for the HRMS application. It is intended for stakeholders and decision-makers — not engineers — and focuses on what technology is being used, why it was chosen, and how it protects and serves the organization.

A separate, detailed technical document exists for the engineering team. This document does not contain code or technical implementation specifics.

2. Technology Approach

The HRMS is being built as a modern, custom web application. All components — the user interface, the business logic engine, and the database — are purpose-built for this system. Nothing is an off-the-shelf product adapted to fit. This gives the organization full control over the system's behaviour, data, and future evolution.

The system is built entirely with open-source, industry-standard technologies that are widely adopted by major global organizations. There is no vendor lock-in — the organization is never dependent on a single commercial software provider.

3. Technology Choices & Rationale

The following table summarizes the key technology choices and the business reason behind each.

Area	Technology Chosen	Why This Was Chosen
User Interface	Next.js 15	A leading web framework used by companies like Netflix, Airbnb, and TikTok. Provides fast page loads, a secure application structure, and a professional user experience.
UI Design System	shadcn/ui + Tailwind CSS	A modern, accessible component library that ensures a consistent, professional look across every screen. Accessibility standards are built in.
Application Language	TypeScript	A strongly-typed programming language that prevents a large class of software bugs at development time — before the software ever reaches users. Reduces defects significantly.
Backend / API	Express.js (Node.js)	A mature, battle-tested API framework used by millions of production applications worldwide. Fast, lightweight, and well-supported.
Database	PostgreSQL	One of the world's most trusted relational databases. Used by Apple, Instagram, Spotify, and thousands of enterprises. Reliable, ACID-compliant, and highly

Area	Technology Chosen	Why This Was Chosen
		capable of complex queries.
Database Access Layer	Prisma ORM	Provides type-safe, secure database access. Eliminates an entire category of security vulnerability (SQL injection) by design. Also manages database migrations cleanly.
Data Validation	Zod	Ensures all data entering the system — from users or other systems — is validated against strict rules before being processed. Prevents bad data from entering the database.
Authentication	JWT (JSON Web Tokens)	Industry-standard secure authentication protocol. Tokens are stored in secure, inaccessible browser storage (httpOnly cookies) — not visible to scripts or browser extensions.
Project Structure	Bun Monorepo	All parts of the system — frontend, backend, shared utilities — live in a single, managed codebase. This ensures consistency, reduces duplication, and simplifies maintenance.

4. System Architecture Overview

The system is structured in three distinct layers, each with a clear responsibility. This separation ensures that changes to one layer do not unexpectedly affect others, and that each layer can be scaled independently as the organization grows.

Layer	What It Does	Technology
Presentation Layer (What users see)	The web application that employees, managers, HR Admins, and Finance Admins interact with in their browser. Handles all screens, forms, dashboards, and reports.	Next.js 15 · shadcn/ui · Tailwind CSS
Application Layer (Business logic)	The API server that processes all requests, enforces business rules, runs the BPV Engine, calculates efficiency scores, and manages all workflows. No business logic exists in the UI layer.	Express.js · TypeScript · Zod
Data Layer (Where data lives)	The database that stores all employee records, configuration entities, asset ownership data, BPV history, rewards, and audit logs. The application layer is the only system that communicates with this layer.	PostgreSQL · Prisma

The three layers communicate in one direction only — the browser talks to the API, and the API talks to the database. The database is never directly accessible from the browser. This is a fundamental security boundary.

Browser → API Server → Database No direct browser-to-database access. Ever.

5. Metadata-Driven Configuration

A core technical decision in this system is that all business rules, scoring formulas, salary structures, and policy thresholds are stored in the database — not written into the application code.

This means that when HR or Finance needs to adjust a policy — for example, changing the BPV weight for certifications, adding a new asset type, or modifying an efficiency tier — they do so through the Admin Dashboard. The change takes effect immediately. No software update, no developer involvement, no system downtime.

What Can Be Changed Without Code	Who Changes It
BPV scoring weights and qualification scores	HR Admin
Salary grade split ratios and salary bands	Finance Admin
Efficiency score weights and incentive tier table	HR Admin
Time reporting block options	HR Admin
Asset types and documentation requirements	HR Admin / IT
Reward formulas and calculation parameters	Finance Admin
Beneficiary types and required documents	HR Admin

What Can Be Changed Without Code	Who Changes It
Session timeout and security thresholds	System Admin

Every configuration change is automatically versioned with a date and the name of the person who made the change. Historical calculations always use the rule that was active at the time — ensuring full auditability.

6. Security Design

Security is built into the system at every layer — it is not an afterthought. The following measures are implemented by design.

Security Area	How It Is Handled
User Authentication	Employees log in with email and password. Passwords are never stored — only a one-way encrypted hash is kept. Login tokens are stored in secure, inaccessible browser storage (httpOnly cookies), not in the browser's readable memory.
Session Management	Login sessions automatically expire after a configurable period (default: 15 minutes of inactivity for the active token). A background refresh token extends the session up to 7 days without requiring re-login, after which the user must log in again.
Role-Based Access Control	Every user has a defined role (Employee, Manager, HR Admin, Finance Admin, System Admin). Each role can only access the data and functions it is authorised for. An employee can only see their own data — never another employee's salary, BPV score, or reward details.
Data Access Boundaries	Salary, BPV scores, and reward data are restricted at the API level — not just hidden in the UI. Even if someone attempted to call the API directly, they would receive an access-denied response.
Audit Trail	Every configuration change, data access event, and administrative action is logged with a timestamp, the user who performed it, and what was changed. Audit logs cannot be edited or deleted.
Input Validation	All data submitted to the system — by users or other systems — is validated against strict rules before being processed. Invalid or malformed data is rejected before it reaches the database.
Database Security	The database is not accessible from the internet. It is accessible only from the application server. All database queries are constructed in a way that prevents SQL injection attacks by design.
Cross-Site Attack Prevention	Technical measures (SameSite cookies, CORS policy, httpOnly flags) prevent common web attack vectors such as Cross-Site Request Forgery (CSRF) and Cross-Site Scripting (XSS).

7. Scalability & Performance

The system is designed to grow with the organization. The following architectural decisions ensure the system remains performant as user numbers, data volumes, and organizational complexity increase.

- The three-layer architecture (UI, API, Database) allows each layer to be scaled independently. If the user count grows significantly, additional API servers can be added without touching the database or the UI.
- All configuration data is cached in application memory after the first read. Subsequent requests for the same configuration are served from cache — reducing database load significantly.
- Database queries are optimized with indexes on all frequently-searched columns (employee status, department, asset type, task status, etc.).
- The Asset Ownership Registry is designed to support up to one million asset records without query performance degradation.
- The API is stateless — it does not store session data on the server. This means multiple API server instances can run in parallel without coordination, enabling horizontal scaling.
- Background jobs (BPV recalculation, efficiency scoring, compliance monitoring, certification expiry alerts) run asynchronously — they do not block user-facing requests.

8. Hosting & Infrastructure

The system is designed for cloud deployment. The specific cloud provider (AWS, Azure, Google Cloud, or a managed VPS) will be confirmed based on organizational preference and data residency requirements.

Component	Hosting Approach
Frontend (Web Application)	Deployed as a static/server-rendered web application. Accessible via a standard web browser on any device — no software installation required for users.
Backend (API Server)	Deployed as a managed Node.js application on the cloud platform of choice. Multiple instances can run in parallel for high availability.
Database (PostgreSQL)	Deployed as a managed PostgreSQL service (e.g., AWS RDS, Azure Database for PostgreSQL, or Supabase). Managed services handle backups, updates, and failover automatically.
File Storage	Employee documents, asset documentation, and certificate uploads are stored in a cloud object storage service (e.g., AWS S3 or Azure Blob Storage) — separate from the database.
Environments	Three environments are maintained: Development (for active development), Staging (for testing and client review), and Production (live system). Changes are promoted through environments before reaching production.

9. Development & Delivery Approach

- The entire codebase — frontend, backend, and shared utilities — is maintained in a single, unified repository (monorepo). This ensures consistency, simplifies dependency management, and makes cross-layer changes straightforward.
- All code changes go through a review process before being merged. No code reaches the production environment without peer review.
- Database schema changes are managed through a migration system — every change to the database structure is tracked, versioned, and reversible.
- The system uses automated type checking across both the frontend and backend. A large class of bugs is caught before code is ever run.
- Shared data validation rules (Zod schemas) are written once and used by both the frontend form validation and the backend API validation — eliminating duplication and ensuring consistency.

10. Document Control

Field	Value
Document Title	HRMS — Technology Design Overview (Client Version)
Version	v1.0
Status	For Review & Approval
Date	July 2026
Classification	Confidential
Companion Documents	SDD v2.0 · FDD v1.0 · TDD v1.0 (Developer Version)
Audience	Client Stakeholders & Decision Makers

The detailed developer-facing Technical Design Document (TDD v1.0) contains all implementation specifics, code conventions, folder structures, and API designs for the engineering team. This document is a summary for stakeholder review and approval.